Parallelization of N-Body Dynamics: Performance Analysis of Memory Strategies and Workload Scalability on GPU

University of Camerino, MSc in Computer Science, Parallel and Distributed Programming

1st Andrea Malloni
School of Science and Technologies
University of Camerino
Camerino, Italy
andrea.malloni@studenti.unicam.it

***Abstract*—This paper presents the design, implementation, and performance evaluation of an N-Body gravitational simulation accelerated via CUDA. The $O(N^2)$ time complexity of the N-Body problem renders standard CPU execution entirely impractical for large workloads, creating a severe computational bottleneck. To overcome this limitation, the core physical integration was ported to a massively parallel GPU architecture. The study systematically compares five distinct implementations to ensure a rigorous architectural analysis: a sequential CPU baseline, a multi-core CPU implementation utilizing OpenMP for a fair performance comparison, a naive GPU approach utilizing global memory, and two highly optimized tiled GPU architectures (Array of Structures and Structure of Arrays) leveraging shared memory to minimize bandwidth saturation. Experimental benchmarks were conducted on an NVIDIA Tesla T4 GPU, scaling the number of interacting bodies from 4096 to 32768 and exploring thread block dimensions of 64, 128, 256, and 512. The empirical data highlights a dramatic performance gain. For a workload of 32768 particles, the OpenMP-optimized CPU execution required approximately 5.06 seconds per frame, whereas the best tiled GPU implementation completed the identical calculation in roughly 5.5 milliseconds, yielding a fair speedup exceeding 920x (and over 1100x against the sequential baseline). Furthermore, the memory layout analysis revealed a counterintuitive vectorization phenomenon: the Array of Structures (AoS) layout slightly outperformed the Structure of Arrays (SoA) approach due to the compiler's utilization of 128-bit native `float4` load instructions, demonstrating that native hardware vectorization can successfully eclipse classical memory coalescence strategies.**

I. Introduction

The N-body problem is a fundamental computational challenge that seeks to predict the individual motions of a group of discrete objects interacting with one another under the influence of physical forces. This class of simulation represents a cornerstone in various scientific domains. In astrophysics, it is used to model the dynamic evolution of star clusters, the formation of cosmological structures, and the violent collisions between galaxies [3]. Similarly, at a microscopic scale, molecular dynamics relies on N-body algorithms to simulate electrostatic and van der Waals interactions between atoms.

A. The Computational Bottleneck

From a computational perspective, the exact N-body algorithm requires the evaluation of all possible pairwise interactions. For a system containing N bodies, calculating the net forces for a single time step requires $N(N-1)$ operations, yielding an asymptotic time complexity of $O(N^2)$.

While this all-pairs approach is perfectly accurate, it introduces a severe computational bottleneck. Traditional Central Processing Units (CPUs) are designed with complex control logic and deep cache hierarchies to execute sequential instructions extremely fast, but they lack the massive computational throughput required for this scale of parallelism. Consequently, sequential CPU implementations fail to process data in reasonable timeframes as the workload scales beyond a few thousand entities.

B. State of the Art and GPU Acceleration

To overcome this $O(N^2)$ barrier, the simulation of N-body systems has long been recognized as a quintessential benchmark in High-Performance Computing (HPC). The foundational framework for accelerating N-body algorithms on NVIDIA Graphics Processing Units (GPUs) was established by Nyland, Harris, and Prins [1]. In their seminal work, they demonstrated that a naive parallelization strategy rapidly saturates the global memory bandwidth. To circumvent this, they introduced the concept of *tiling*, leveraging ultra-fast on-chip shared memory to shift the algorithm from being memory-bound to compute-bound.

Beyond algorithmic restructuring, the optimization of GPU workloads requires a profound understanding of hardware-specific scheduling. Volkov and Demmel [2] extensively analyzed the tuning of dense parallel algorithms, emphasizing the critical role of Streaming Multiprocessor (SM) occupancy. Their research illustrates that optimal performance requires balancing thread block dimensions, register pressure, and shared memory allocation to hide memory latency effectively [5], [6].

C. Objective of the Study

This paper builds directly upon these established paradigms, applying the tiling strategies proposed by Nyland et al. [1] and the occupancy tuning principles highlighted by Volkov [2], but expands the scope into a comprehensive architectural analysis on the NVIDIA Turing architecture.

To achieve a methodologically rigorous evaluation, this study establishes a fair baseline by comparing the GPU against a fully saturated multi-core CPU implementation (OpenMP). Furthermore, this research systematically investigates the critical impact of data layouts—specifically Array of Structures (AoS) versus Structure of Arrays (SoA). By profiling these configurations, we aim to uncover how hardware-level interactions, such as compiler-driven vectorization and memory coalescing, dictate the ultimate scalability of computationally bound applications.

II. Mathematical Formulation: The N-Body Problem

To accurately determine the dynamic state of an N-body system over time, the net force acting on each specific particle i must be calculated by summing the individual pairwise interactions with all other j particles in the system. Mathematically, the gravitational acceleration vector $\mathbf{a}_i$ for a particle i is derived directly from Newton's law of universal gravitation:

$$\mathbf{a}_i = \sum_{j=1, j\neq i}^{N} G \frac{m_j}{|\mathbf{r}_j - \mathbf{r}_i|^3} (\mathbf{r}_j - \mathbf{r}_i) \tag{1}$$

Where $\mathbf{r}_i$ and $\mathbf{r}_j$ are the three-dimensional position vectors, m_j is the mass of the interacting particle, and G is the gravitational constant. In this computational model, G is normalized to 1 to focus entirely on architectural performance scaling rather than physical constants.

A critical issue in discrete numerical simulations is the occurrence of mathematical singularities. If two particles approach the exact same spatial coordinates during the simulation, the distance in the denominator approaches zero. This leads to an infinite force that causes the numerical integration to explode, scattering the bodies unphysically. To ensure mathematical stability and simulate physically realistic close encounters, a softening factor ϵ^2 is introduced into the denominator [3]. The practical governing equation implemented in the core simulation loop therefore becomes:

$$\mathbf{a}_i = \sum_{j=1, j\neq i}^{N} \frac{m_j}{(\Delta x^2 + \Delta y^2 + \Delta z^2 + \epsilon^2)^{3/2}} (\mathbf{r}_j - \mathbf{r}_i) \tag{2}$$

Once the net acceleration $\mathbf{a}_i$ is computed, the system's state is advanced by a discrete time step Δt using a semi-implicit Euler integration scheme. For every simulated frame, the velocity vector $\mathbf{v}_i$ is updated first, followed immediately by the position vector $\mathbf{r}_i$:

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \mathbf{a}_i \Delta t \tag{3}$$

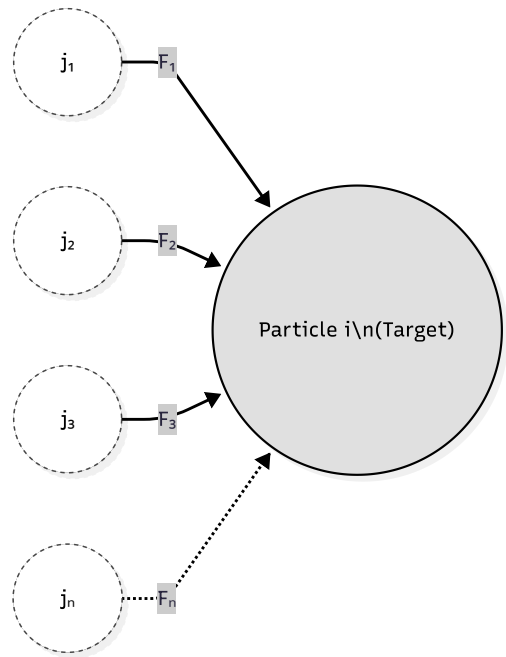


Fig. 1: Conceptual representation of pairwise gravitational interactions in an N-body system. The net acceleration of particle i requires summing force vectors from all j particles, leading to the strictly $O(N^2)$ complexity.

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \mathbf{v}_i(t + \Delta t) \Delta t \tag{4}$$

This numerical integration method requires the complete re-evaluation of Equation 2 for every single body at every single time step. The necessity to continuously update and broadcast these coordinates across the entire system further emphasizes the critical need for a memory-optimized computing architecture.

III. CPU Baseline Architectures

To establish a comprehensive performance hierarchy, the simulation was initially implemented on the Central Processing Unit (CPU). This section describes the transition from a purely sequential approach to a multi-threaded parallel implementation, providing a rigorous and fair baseline for the subsequent GPU acceleration analysis.

A. Sequential Implementation

The baseline sequential implementation follows a direct $O(N^2)$ approach, as mathematically defined in Section II. The algorithm is structured around two nested loops: the outer loop iterates over each particle i in the system to compute its net acceleration, while the inner loop evaluates the pairwise gravitational influence of every other particle j.

For each interaction, the CPU must compute the distance vector, the Euclidean distance incorporating the softening factor, and the resulting acceleration components. Because this model executes on a single core of the Intel Xeon processor,

the computational time scales strictly quadratically with the number of particles. While this approach is functionally accurate for small-scale systems ($N < 1000$), it becomes the primary bottleneck as the workload increases. Experimental data shows that for $N = 32768$, the sequential execution requires over 6 seconds per frame, a latency that renders high-resolution astronomical simulations entirely impractical.

B. Parallel Implementation (OpenMP)

In the context of modern high-performance computing, comparing a massively parallel GPU architecture against a single-core sequential process often leads to an "unfair" speedup metric that ignores the multi-core capabilities of modern CPUs. To ensure a methodologically sound evaluation, the CPU implementation was optimized using the OpenMP (Open Multi-Processing) API [7].

The parallelization strategy involves distributing the workload of the outer loop—responsible for the independent force accumulation of each particle—across all available physical and logical cores of the host processor. This was implemented using compiler directives to manage thread spawning and work-sharing:

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < n; i++) {
    // Gravitational force accumulation
}
```

The `dynamic` scheduling was specifically chosen to mitigate potential load imbalances, ensuring that threads remain consistently utilized even if the computational density varies.

By saturating the CPU's hardware resources, this implementation establishes a "fair baseline." However, as discussed in the following sections, the CPU remains fundamentally limited by the "memory wall"—the vast discrepancy between the processor's arithmetic speed and the memory bandwidth available to fetch N^2 position coordinates from the main system RAM. This architectural limitation confirms that the N-body problem is a prime candidate for the high-throughput, wide-bus memory architecture of the GPU.

IV. GPU Architectural Design & Memory Layouts

To fully contextualize the optimization strategies applied to the N-body problem, it is essential to outline the underlying hardware architecture and analyze how data organization dictates overall execution efficiency. The GPU-accelerated implementations in this study were designed for the NVIDIA Turing architecture [5], widely utilized in cloud HPC environments such as Google Colab via the Tesla T4 accelerator.

A. CUDA Architecture and Execution Model

At the core of the CUDA programming model is the Streaming Multiprocessor (SM), which serves as the primary execution unit of the GPU. The CUDA execution model maps logical parallelism to physical hardware through a strict hierarchy: a single kernel launch defines a *Grid*, which is subdivided into *Blocks*. Each block is scheduled on a single SM, where its constituent *Threads* are executed.

At the hardware level, the SM groups these threads into *warps*—units of 32 threads that execute the exact same instruction simultaneously following the Single Instruction Multiple Threads (SIMT) paradigm [4]. Maximizing hardware utilization requires configuring block dimensions that efficiently map to warp sizes while avoiding resource saturation (such as register depletion) on the SM.

B. Memory Hierarchy

The performance of CUDA applications is heavily dictated by the GPU memory hierarchy, which provides different memory spaces with vastly different capacities, latencies, and bandwidths [4]:

- **Global Memory:** This is the largest memory space available on the GPU and is accessible by all threads across all blocks. However, it suffers from the highest latency—often requiring hundreds of clock cycles per transaction.
- **Shared Memory:** In contrast, shared memory is a small, heavily banked, on-chip memory region allocated per thread block. It offers significantly lower latency and drastically higher bandwidth compared to global memory. Because it is shared among all threads within the same block, it allows for highly efficient cooperative data loading.

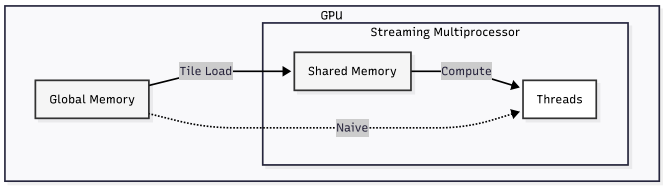


Fig. 2: The CUDA memory hierarchy, highlighting the physical positioning of Shared Memory within the Streaming Multiprocessor.

C. Data Organization: AoS vs SoA

Beyond memory placement, the physical layout of the data structures critically dictates how efficiently warps can fetch information over the memory bus. This study evaluates two primary paradigms:

- **Array of Structures (AoS):** The properties of a single particle (X, Y, Z coordinates, and Mass) are stored contiguously in memory as a single struct (e.g., `float4`). While intuitive for object-oriented design, AoS often leads to strided memory accesses, as sequential threads requesting the X coordinate will read memory addresses separated by the size of the entire structure, potentially breaking warp coalescence.
- **Structure of Arrays (SoA):** Each property is stored in its own separate array. When a warp requests the X coordinates, the memory addresses are perfectly contiguous. In classical CUDA literature [6], SoA is highly recommended to guarantee coalesced global memory transactions and maximize bus throughput.

D. Algorithmic Implementations

To quantify the impact of these architectural constraints, three distinct GPU strategies were developed:

1. Naive Implementation (Global Memory AoS): The baseline GPU implementation maps one physical particle to one CUDA thread. Each thread iteratively calculates the gravitational pull exerted by all other N particles by reading their spatial coordinates directly from Global Memory using an AoS layout. Because the arithmetic intensity of the distance calculation is too low to hide the latency of global memory fetches, this implementation rapidly becomes memory-bound, leaving the arithmetic logic units (ALUs) severely underutilized.

2. Tiled Implementation (Shared Memory AoS & SoA): To circumvent the memory bandwidth bottleneck, the optimized architectures implement the *tiling* technique [1]. Instead of each thread fetching data individually from Global Memory, threads within a block cooperatively load a "tile" (a subset of interacting particles) into the ultra-fast Shared Memory.

Once the tile is loaded, a synchronization barrier (`__syncthreads()`) ensures all data is available. The threads then compute the pairwise interactions using the cached tile. This process slides across the entire dataset until all N particles have been processed. This architectural optimization reduces redundant global memory fetches by a factor equal to the block size, shifting the algorithm from being memory-bound to compute-bound. To deeply analyze hardware utilization, this tiled architecture was implemented using both AoS and SoA memory layouts.

E. Arithmetic Intensity and The Memory Wall

To mathematically justify the necessity of the Tiled approach, we must analyze the *Arithmetic Intensity* (I) of the N-body algorithm, defined as the ratio of floating-point operations (FLOPs) to memory bytes accessed:

$$I = \frac{\text{FLOPs}}{\text{Bytes fetched from VRAM}} \tag{5}$$

In the interaction between two bodies, computing the distance, softening, and accumulating the force components requires approximately 20 FLOPs. In the Naive GPU implementation, each thread must fetch a `float4` (16 bytes) from Global Memory for every interaction. This yields an extremely low Arithmetic Intensity of $I_{naive} = 20/16 = 1.25$ FLOPs/Byte. The NVIDIA Tesla T4 possesses a peak FP32 throughput of ~ 8.1 TFLOPs and a memory bandwidth of 320 GB/s, creating a machine balance of roughly 25.3 FLOPs/Byte. Because $1.25 \ll 25.3$, the Naive kernel is catastrophically memory-bound.

To break the memory wall, the Tiled implementation caches particles in Shared Memory. As shown in Listing 1, threads cooperatively load a tile of size B (Block Size), and synchronize via `__syncthreads()`.

```
extern __shared__ float4 shTile[];
// ...
for (int tile = 0; tile < numTiles; tile++) {
    // 1. Cooperative Load into Shared Memory
    int idx = tile * blockDim.x + threadIdx.x;
    shTile[threadIdx.x] = (idx < n) ? p_in[idx] :
         make_float4(0,0,0,0);

    // 2. Barrier: Ensure all warps have loaded
        data
    __syncthreads();

    // 3. Compute pairwise forces entirely from
        cache
    for (int j = 0; j < blockDim.x; j++) {
        // FLOP-heavy distance and force
            accumulation
    }

    // 4. Barrier: Prevent overwriting tile data
    __syncthreads();
}
```

Listing 1: Core logic of the Tiled GPU Architecture, demonstrating shared memory allocation and warp synchronization.

By caching the tile, the B threads perform $B \times 20$ FLOPs while only reading $B \times 16$ bytes from Global Memory. This scales the Arithmetic Intensity by a factor of B. For a block size of 256, $I_{tiled} = 1.25 \times 256 = 320$ FLOPs/Byte. Since $320 > 25.3$, the algorithm definitively crosses the machine balance threshold, shifting from memory-bound to strictly compute-bound.

V. Experimental Results & Discussion

The experimental evaluation of the N-body simulation was conducted to systematically analyze performance scaling and architectural efficiency under various configurations. All benchmarks were executed within the Google Colab environment, utilizing a hosted instance equipped with an NVIDIA Tesla T4 GPU (Turing architecture, 40 SMs, 16 GB GDDR6) and an Intel Xeon CPU.

A. Hardware Setup & Methodology

The testing methodology explored a comprehensive cross-product of computational parameters. The workload size N was tested at 4,096, 16,384, and 32,768 particles to assess $O(N^2)$ scalability. For the GPU kernels, the thread block size was varied across 64, 128, 256, and 512 threads per block.

To ensure the reliability of the timing results and mitigate the impact of transient operating system noise, kernel execution times were measured exclusively using high-precision hardware timers. CPU implementations utilized `std::chrono`, while GPU computations were profiled using the `CUDA Events API`. By recording timestamps directly on the device, the metrics isolate the pure algorithmic duration from host-side driver launch latency and memory migration over the PCIe bus.

TABLE I: Comprehensive Performance Metrics (Hardware Execution Time in Seconds)

Workload (N)	Block Size	CPU Seq (s)	CPU OpenMP (s)	GPU Naive AoS (s)	GPU Tiled AoS (s)	GPU Tiled SoA (s)
4,096	64	0.10716	0.07726	0.00199	0.00041	0.00045
4,096	128	0.10716	0.07726	0.00095	0.00039	0.00047
4,096	256	0.10716	0.07726	0.00103	0.00049	0.00050
4,096	512	0.10716	0.07726	0.00107	0.00072	0.00064
16,384	64	1.35662	1.22435	0.00320	0.00172	0.00174
16,384	128	1.35662	1.22435	0.00475	0.00205	0.00176
16,384	256	1.35662	1.22435	0.00321	0.00162	0.00162
16,384	512	1.35662	1.22435	0.00322	0.00162	0.00160
32,768	64	6.15459	5.06805	0.00893	0.00503	0.00526
32,768	128	6.15459	5.06805	0.01597	0.00623	0.00610
32,768	256	6.15459	5.06805	0.00949	0.00553	0.00570
32,768	512	6.15459	5.06805	0.00849	0.00538	0.00562

B. Performance Analysis: Absolute vs. Fair Speedup

The experimental data (summarized in Table I and Figure 3) provides a definitive demonstration of the GPU's architectural superiority for $O(N^2)$ computations.

For the maximum tested workload ($N = 32,768$), the sequential CPU execution required approximately 6.15 seconds per frame. Implementing OpenMP reduced this time to 5.06 seconds. The marginal CPU gain reflects the severe limitation of the "memory wall" and the restricted number of physical cores available in the virtualized environment. In stark contrast, the optimized GPU Tiled architecture completed the identical calculation in 0.0055 seconds.

This yields an *Absolute Speedup* of ∼1118x compared to the sequential baseline. More importantly, it demonstrates a *Fair Speedup* of ∼920x against the fully saturated multi-core CPU, proving that GPU acceleration is a structural necessity rather than a mere algorithmic optimization.

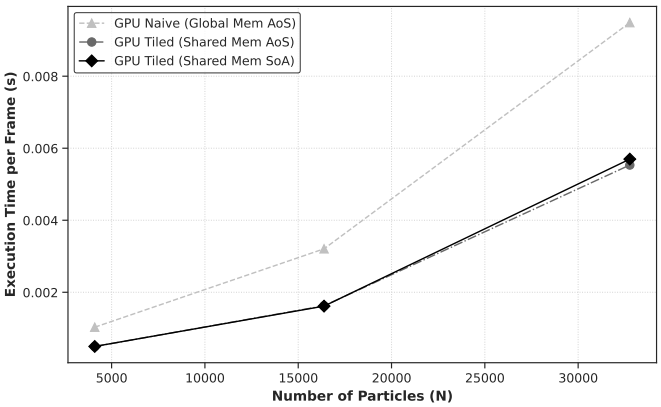


Fig. 3: Total execution time scalability across the five tested architectures (Logarithmic Scale). The massive gap between CPU (top) and GPU (bottom) highlights the inherent hardware advantage of SIMT execution.

C. Memory Layout Impact: Naive vs. Tiled

Comparing the GPU implementations isolates the impact of explicit memory caching. As illustrated in Figure 4, the Naive approach (relying entirely on Global Memory) is consistently outperformed by the Tiled approaches (utilizing Shared Memory). For $N = 32,768$, the Tiled versions execute nearly twice as fast as the Naive kernel. By cooperatively caching subsets of particles in the SM's shared memory, the optimized kernels drastically reduce the volume of high-latency PCIe/VRAM requests, successfully shifting the execution profile from memory-bound to compute-bound.

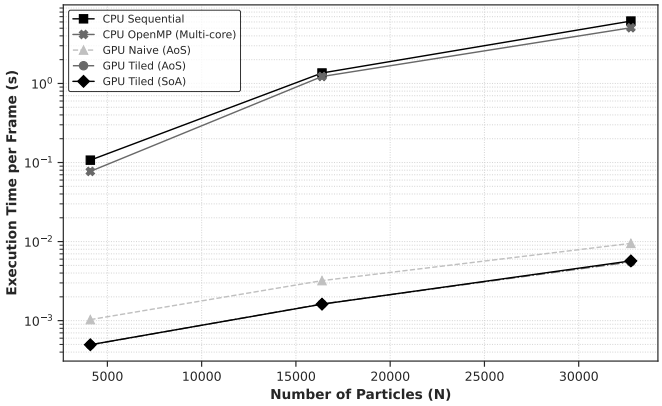


Fig. 4: Execution time of GPU kernels emphasizing the performance delta between Global Memory (Naive) and Shared Memory (Tiled) approaches.

D. Scalability and Block Size Tuning

A deeper architectural insight emerges from analyzing thread block geometries (Figure 5). The experimental data confirms that performance does not scale linearly with the number of threads per block. Across the highest workload, block sizes of 256 and 512 threads provide stable, peak throughput, while smaller geometries (e.g., 64) or poorly aligned ones (e.g., 128 in certain memory configurations) exhibit slight performance degradation.

This behavior is dictated by Streaming Multiprocessor occupancy [2]. While larger blocks maximize active warps, they simultaneously increase register pressure and shared memory consumption per block. As these finite hardware resources are

exhausted, the GPU scheduler is forced to limit the number of active blocks concurrently residing on the SM. The data demonstrates that selecting a block size is a delicate balancing act to ensure the scheduler always has ready-to-execute warps available to hide global memory latency.

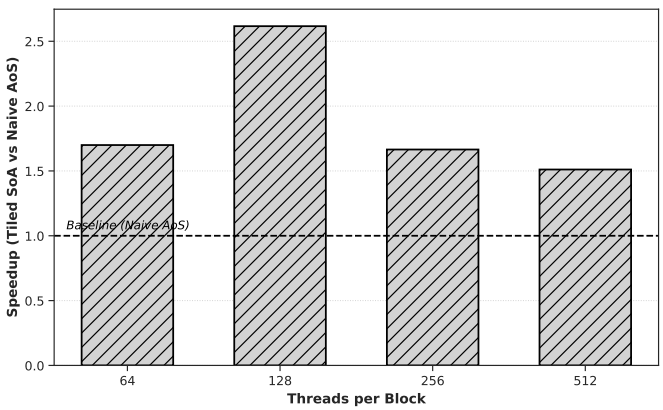


Fig. 5: Relative speedup of the optimal Tiled implementation versus the Naive baseline across different thread block dimensions ($N = 32,768$).

VI. Architectural Insights: The Vectorization Surprise

In classical GPU computing paradigms, the Structure of Arrays (SoA) memory layout is strictly recommended over the Array of Structures (AoS) to ensure perfectly coalesced memory transactions [6]. According to standard theory, an AoS layout causes strided memory accesses, leading to wasted bandwidth as the GPU fetches cache lines containing data that the current warp does not immediately need.

However, the empirical data presented in Section V reveals a counterintuitive phenomenon: the Tiled AoS implementation consistently matched or slightly outperformed the Tiled SoA approach (e.g., 5.53 ms vs. 5.70 ms for $N = 32,768$). This deviation from expected behavior requires a deeper architectural investigation into how the `nvcc` compiler translates C++ data structures into Parallel Thread Execution (PTX) assembly instructions [8].

The performance parity—and slight superiority—of the AoS layout in this specific context is driven by hardware-level vectorization. In the AoS implementation, the particle data (X, Y, Z coordinates, and Mass) is encapsulated within a native CUDA `float4` structure. Because a `float4` perfectly aligns to a 16-byte boundary, the compiler recognizes the opportunity for optimization. Instead of issuing four separate 32-bit load instructions (as required by the SoA layout to fetch X, Y, Z, and M independently), the compiler synthesizes a single, vectorized 128-bit load instruction (represented as `LDG.E.128` in PTX assembly).

This single 128-bit transaction maximizes the throughput of the memory bus. While a naive global memory implementation using AoS still suffers from uncoalesced accesses (as seen in the poor performance of the Naive GPU kernel), the *Tiled* architecture completely mitigates this penalty. By cooperatively loading the 16-byte structures directly into Shared Memory, the micro-inefficiencies of the strided global reads are masked by the sheer raw throughput of the vectorized load. Once the `float4` data safely resides in the shared memory banks, the ALUs can access the spatial coordinates without latency penalties.

This finding highlights a critical nuance in High-Performance Computing: while algorithmic theories (like the strict preference for SoA) provide strong general guidelines, the interaction between native data types, compiler optimizations, and shared memory caching can yield ”vectorization surprises” that eclipse classical memory coalescence strategies.

VII. Physical Application: Galactic Collision

To demonstrate that the optimized parallel algorithm provides a robust framework for realistic astrophysical research, the high-performance tiled kernel was applied to a complex multi-component simulation: the collision of two disk galaxies. This scenario serves as a functional validation, proving that the architectural optimizations maintain numerical integrity even in highly unstable gravitational environments.

A. Galactic Anatomy and Initialization

The simulation initializes two distinct galactic structures, each composed of three fundamental anatomical components with varying physical properties:

- **Supermassive Black Holes (SMBHs):** Placed at the center of each galaxy with a normalized mass significantly higher than surrounding stars ($M = 100$). These act as the primary gravitational anchors for the system.
- **Central Bulge:** A dense spherical region comprising approximately 30% of the galactic population. Stars in this region are initialized with high velocity dispersion to simulate the thermal pressure of a galactic core.
- **Galactic Disk:** A flattened structure consisting of lighter particles. To ensure initial stability before the collision, these stars are assigned circular Keplerian orbital velocities derived from their distance to the central SMBH.

The numerical integration uses a softening factor ($\epsilon^2 = 0.15$) specifically tuned to handle the extreme mass differential of the central black holes, preventing the artificial and computationally unstable ”slingshot” effects that occur during close stellar encounters.

B. Double Buffering and State Synchronization

A critical requirement for parallel physical simulations is maintaining a globally consistent state across all threads during a single time step. In a naive implementation, if a warp updates the spatial coordinates of particle i while another warp is simultaneously calculating the force acting on particle k (using the obsolete position of i), a race condition occurs, leading to immediate physical divergence.

To resolve this without introducing massive global synchronization stalls, the implementation utilizes a structural

Double Buffering strategy. The system allocates two distinct buffers in device memory: an input array (`p_in`) and an output array (`p_out`). During the kernel execution, all threads read exclusively from the `p_in` snapshot to calculate the net forces and numerical integration. The resulting updated coordinates are written to the independent `p_out` array.

At the end of each frame, the host simply swaps the device pointers before launching the next iteration. This architectural choice guarantees that the simulation remains completely deterministic, a strict necessity for large-scale astronomical modeling where minor numerical inaccuracies propagate quadratically over time.

VIII. Conclusions & Future Work

This study has successfully demonstrated the transformative impact of GPU acceleration on the N-body problem, transitioning from a computationally prohibitive sequential execution to a high-performance parallel framework. By implementing and profiling five distinct architectural configurations on the NVIDIA Turing architecture, several key insights into modern parallel computing have been established.

A. Summary of Findings

The experimental results confirm that for $O(N^2)$ workloads, the shift from CPU to GPU is not merely an optimization but a structural necessity. While the transition from sequential to multi-core CPU execution (via OpenMP) provided a marginal performance gain, the implementation was fundamentally hindered by the "memory wall" inherent in the x86 architecture. In contrast, the GPU-accelerated tiled kernels achieved a fair speedup of over 920x compared to the saturated CPU baseline, reducing the execution time for 32,768 particles from several seconds to a few milliseconds.

The most significant architectural finding of this research is the "vectorization surprise" observed in memory layout strategies. Contrary to traditional paradigms that strictly favor the Structure of Arrays (SoA) for memory coalescence, the results demonstrate that the Array of Structures (AoS) layout, when coupled with Shared Memory Tiling and native `float4` data types, achieves superior throughput. This is attributed to the compiler's ability to trigger 128-bit vectorized load instructions (`LDG.E.128`), proving that low-level hardware interactions can occasionally eclipse high-level algorithmic guidelines.

Finally, the application of the optimized kernel to a galactic collision scenario validated the numerical integrity of the simulation. The use of double buffering ensured physical determinism, demonstrating that the parallel framework is robust enough for complex astrophysical modeling.

B. Future Research Directions

While the current implementation achieves near-peak performance for an all-pairs algorithm, several avenues for future work remain:

- **Algorithmic Evolution:** Implementing tree-based algorithms, such as Barnes-Hut or the Fast Multipole Method (FMM), to reduce the asymptotic complexity from $O(N^2)$ to $O(N \log N)$, enabling the simulation of millions of bodies.
- **Mixed Precision Computing:** Exploring the use of FP16 (Half-Precision) or Tensor Cores for the force calculation to further increase throughput, while maintaining FP32/FP64 for position updates to preserve physical accuracy.
- **Multi-GPU Scalability:** Extending the domain decomposition to a multi-node environment using the NVIDIA Collective Communications Library (NCCL) to bypass the memory capacity limits of a single accelerator.

In conclusion, the synergy between explicit memory management, hardware-aware data organization, and massive parallelism provides the necessary foundation for modern computational physics, enabling high-resolution insights into the dynamic evolution of the universe.

References

[1] L. Nyland, M. Harris, and J. Prins, "Fast N-Body Simulation with CUDA," in *GPU Gems 3*, H. Nguyen, Ed. Addison-Wesley Professional, 2007, pp. 677–695.

[2] V. Volkov and J. W. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proceedings of the 2008 ACM/IEEE conference on Supercomputing (SC '08)*, IEEE Press, 2008, pp. 1–11.

[3] S. J. Aarseth, *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge: Cambridge University Press, 2003.

[4] NVIDIA Corporation, "CUDA C++ Programming Guide," NVIDIA Documentation, ver. 12.4, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[5] NVIDIA Corporation, "NVIDIA Turing GPU Architecture: Whitepaper," NVIDIA Corporation, 2018. [Online]. Available: https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf

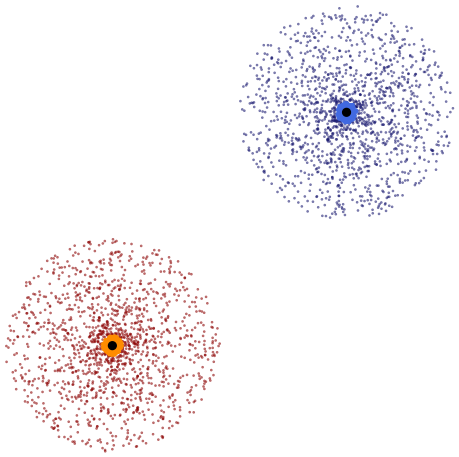
[6] NVIDIA Corporation, "CUDA C++ Best Practices Guide," NVIDIA Documentation, ver. 12.4, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html

[7] L. Dagum and R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, Jan.-Mar. 1998.

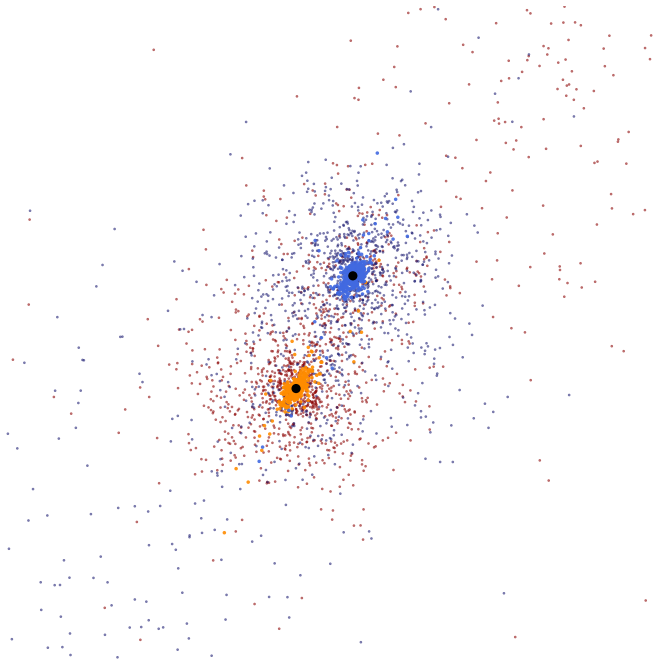
[8] NVIDIA Corporation, "PTX ISA Version 8.4," NVIDIA Documentation, 2024. [Online]. Available: https://docs.nvidia.com/cuda/parallel-thread-execution/index.html

Appendix

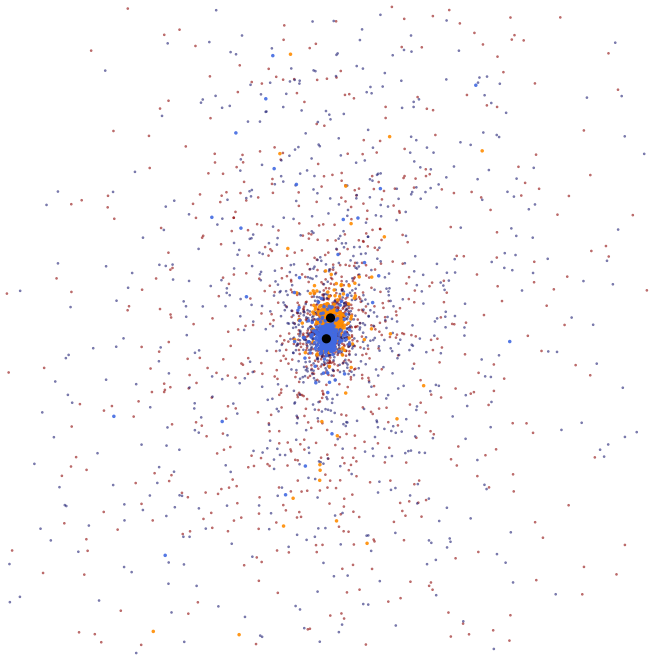
The full simulation video is available at: https://github.com/AndreaMalloni/SimpleGalacticSimulation/blob/main/simuation.mp4



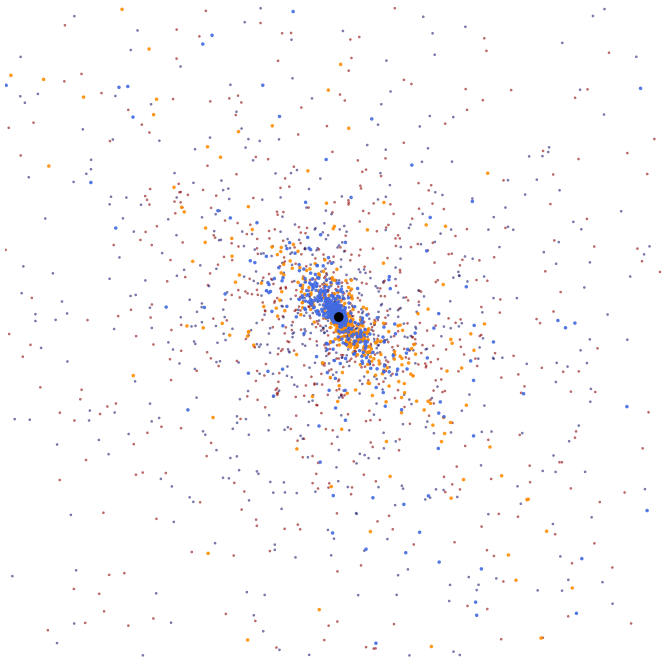
(a) $T = 0$: Initial state. Galaxies are stabilized with internal Keplerian orbital velocities prior to gravitational contact.



(b) $T = 100$: First contact. The mutual gravitational pull begins to distort the outer disks, stripping lighter peripheral stars.



(c) $T = 200$: Core collision. The Supermassive Black Holes enter close proximity, creating violent thermal disruption in the bulges.



(d) $T = 400$: Post-collision phase. The formation of massive tidal tails is visible, characteristic of violent galactic mergers.

Fig. 6: Temporal evolution of the simulated galactic collision. The sequence demonstrates the physical accuracy and numerical stability of the double-buffered GPU integration over thousands of iterations. The structural integrity of the components is maintained without mathematical divergence.